CAIRO UNIVERSITY
Faculty of Computers and Artificial Intelligence

CS213: Object-Oriented Programming (Programming II)
Academic Year 2026-2027 | First Semester

Vole Machine Simulator OOP Design Report

Assignment 2 (Version 4.0) — Part 1: OOP Design with AI
Course instructor: Dr. Mohammad El-Ramly

Student name	Youssef Ahmed AbdElNabi Ahmed Sherif
Student ID	20250853
Section	3
Email	mister.youssef.sherif@gmail.com
Phone (team leader)	01091931084 (individual submission; I am the only team member)
University	Cairo University
Faculty	Faculty of Computers and Artificial Intelligence
Course	CS213 — Object-Oriented Programming
Assignment	Assignment 2, Part 1 (Task 1: OOP design) — Version 4.0
Date	26 October 2026
GitHub repository	https://github.com/Mr-Youssef-Sherif/vole-machine

Work Breakdown (who did what)

Task	Done by
Studying the Vole machine and the assignment specification	Youssef Ahmed AbdElNabi Ahmed Sherif (20250853)
Writing the two sample Vole programs	Youssef Ahmed AbdElNabi Ahmed Sherif (20250853)
Initial design (classes, PlantUML, UML diagram)	Youssef Ahmed AbdElNabi Ahmed Sherif (20250853)
Prompts and chats with the three AIs (Gemini, ChatGPT, DeepSeek)	Youssef Ahmed AbdElNabi Ahmed Sherif (20250853)
Comparing the designs and selecting the best one	Youssef Ahmed AbdElNabi Ahmed Sherif (20250853)
Final design dialogue with the fourth AI (Claude)	Youssef Ahmed AbdElNabi Ahmed Sherif (20250853)
Report writing and video	Youssef Ahmed AbdElNabi Ahmed Sherif (20250853)

This was an individual submission. I did all of the work above alone; there are no other team members.

Table of Contents

1. Introduction **4**

1.1 The Vole machine . . . 4

1.2 Architecture . . . 4

1.3 Instruction set (2026/27 version) . . . 4

1.4 What the simulator must do (Task 1) . . . 4

1.5 Process followed . . . 5

2. Initial Design **6**

2.1 Overview . . . 6

2.2 Classes and responsibilities . . . 6

2.3 Relationships . . . 6

2.4 Open questions I deliberately left for the AI review . . . 6

2.5 PlantUML code . . . 7

3. AI Designs **9**

3.1 Google Gemini . . . 9

3.2 OpenAI ChatGPT . . . 12

3.3 DeepSeek . . . 15

3.4 Comparison of the four designs . . . 19

4. Final Design **20**

4.1 How the final design was reached . . . 20

4.2 Final classes . . . 20

4.3 Instruction field mapping (checked against the specification examples) . . . 21

4.4 Execution workflow . . . 21

4.5 Error handling . . . 21

4.6 Key design decisions . . . 22

4.7 Corrections made after checking the official specification . . . 23

4.8 Specified versus assumed . . . 23

4.9 Why this is the best design . . . 24

4.10 Points to confirm with the instructor . . . 24

4.11 Final PlantUML code . . . 25

4.12 What I learned . . . 28

5. Examples **29**

5.1 Program 1 — counter from 0 to 10 . . . 29

5.2 Program 2 — reverse counter from 10 to 0 . . . 29

6. Video Link **30**

Appendix A: Repository structure 30

Appendix B: References 30

Project Links

Item	Link
GitHub repository (code, prompts, chats, diagrams, programs)	https://github.com/Mr-Youssef-Sherif/vole-machine
Google Gemini chat	https://share.gemini.google/Cu4BA2LyB5Cs
OpenAI ChatGPT chat	https://chatgpt.com/share/6ade6ac8-aca0-83ea-8fd5-b08bdd2325d6
DeepSeek chat	https://chat.deepseek.com/share/kqc985rblp5l3l6vh0
Claude final-design chat	https://claude.ai/share/b2b07e58-df8e-465e-9e09-83bb4fb49743
Presentation video (Google Drive folder)	https://drive.google.com/drive/folders/1kHbSSZJQRunnAbgXUU8N98l1X9Rz36o9?usp=drive_link

The full conversations are available through the chat links above. The repository also contains the saved prompts and answers (`ai-conversations/`), the PlantUML sources and rendered diagrams of every design, and the sample programs (see Appendix A).

1. Introduction

1.1 The Vole machine

The Vole machine is a deliberately small, fictitious computer that J. Glenn Brookshear uses in *Computer Science: An Overview* to explain how a CPU runs a stored program. The assignment is to design an object-oriented simulator for it. This is a **design-only** task (UML class diagram, no implementation).

1.2 Architecture

- **CPU:** 16 general-purpose 8-bit registers (R0 to RF), a Program Counter (PC) holding the byte address of the next instruction, and a 16-bit Instruction Register (IR) holding the instruction being executed.
- **Main memory:** 256 byte-addressable cells, addresses 00 to FF. An instruction is 16 bits (four hex digits) and so occupies **two consecutive cells**; the first cell holds the high byte. A program is loaded at an **even** start address below FF.
- **Screen:** in this semester's version, address **FF is memory-mapped to the screen**: storing a value to FF writes it to the screen.
- **Machine cycle:** *fetch* (read two bytes into IR, advance PC by 2), *decode* (split the four hex digits into opcode and operands, check that it is valid) and *execute*. A jump works by copying XY into PC during execute. Execution stops at the halt instruction.

1.3 Instruction set (2026/27 version)

In the formats below, R, S and T are register numbers (one hex digit each) and XY is an 8-bit value or a memory address (two hex digits).

Format	Meaning	Example (from the assignment)
`1RXY`	LOAD register R from the memory cell at address XY	`14A3`: cell A3 into R4
`2RXY`	LOAD register R with the value XY	`20A3`: R0 = A3
`3RXY`	STORE register R into the memory cell at address XY	`35B1`: R5 into cell B1
`3RFF`	STORE to FF, which is the screen (writing to FF is writing to the screen)	any store with XY = FF
`40RS`	MOVE the pattern in register R to register S	`40A4`: copy RA into R4
`5RST`	ADD S and T as two's-complement values, result in R	`5726`: R7 = R2 + R6
`6RST`	ADD S and T as floating-point values, result in R	`634E`: R3 = R4 + RE (float)
`7RST`	OR of S and T into R	`7CB4`: RC = RB OR R4
`8RST`	AND of S and T into R	`8045`: R0 = R4 AND R5
`9RST`	XOR of S and T into R	`95F3`: R5 = RF XOR R3
`AR0X`	ROTATE register R right X times (circular)	`A403`: rotate R4 right 3 bits
`BRXY`	JUMP to address XY if register R equals register R0; otherwise continue	`B43C`: if R4 == R0 then PC = 3C
`C000`	HALT execution	`C000`

1.4 What the simulator must do (Task 1)

- Offer a menu; load a program from a file into an even memory address (below FF); let the user overwrite any register or memory cell.
- Set PC to the program's start; display memory as a matrix, plus IR, PC and the 16 registers.

- Fetch into IR, validate, decode, execute and update PC — either all at once or step by step (press Enter for each step); show the state and the screen after the run (or after each step).
- Ask whether to exit or load another program. Program files hold one four-hex-digit instruction per line; blank lines and lines starting with ; are ignored and the rest of an instruction line is ignored. Programs with invalid instructions must be rejected.

1.5 Process followed

Step	What I did	Result in the repository
1	Studied the Vole machine (Brookshear pp. 73-93, simulators) and the assignment.	`CS213-2026-2027-Assignment2-V4.0.pdf`
2	Wrote two sample programs (a counter and a reverse counter).	`programs/`
3	Wrote my own initial design and its PlantUML.	`initial design/`
4	Gave three different AIs (Gemini, ChatGPT, DeepSeek) the machine description, the task and my initial design, separately. Each produced a design and, after a follow-up prompt, a critically revised design with PlantUML.	`ai-conversations/`
5	Compared my design and the three revised AI designs (strengths, weaknesses, incorrect assumptions) and picked the strongest base.	Section 3
6	Asked a fourth AI (Claude) to compare all designs critically and produce a final design, then to re-check that design against the official specification. This second pass found real problems that were fixed.	`ai-conversations/final revision/`
7	Recorded the report and the presentation video.	This report; Section 6

2. Initial Design

2.1 Overview

My initial design separates the simulator into components with distinct responsibilities. A central **VoleMachine** coordinates the system while the CPU, memory, register file, instruction decoder, program loader and output device each do one job. The goals were encapsulation, modularity, separation of responsibilities and maintainability, while avoiding needless classes. I deliberately used plain arrays for the 256 memory bytes and 16 registers instead of one object per cell or register, because the elements are identical and have no behaviour of their own.

2.2 Classes and responsibilities

Class	Responsibility	Key attributes	Key methods
VoleMachine	Coordinates the simulator and its lifecycle; single entry point.	cpu, memory, registerFile, outputDevice, programLoader, isInitialized	initialize, reset, loadProgram, run, step, getMachineState
Memory	Stores instructions and data in 256 byte-addressable cells; validates addresses; writes to FF trigger output.	cells[256], memorySize, outputAddress (FF)	read, write, loadBytes, clear, isValidAddress
RegisterFile	Holds the 16 general-purpose registers R0-RF. R0 is a normal register (not hard-wired to zero).	registers[16], registerCount	read, write, clear, isValidRegister
CPU	Runs the fetch-decode-execute cycle; owns PC, IR and the halted flag; handles jumps and halt.	programCounter, instructionRegister, isHalted	fetch, decode, execute, step, run, updateProgramCounter, halt, reset
InstructionDecoder	Splits an instruction into opcode and operands and validates it (including special forms 40RS, AR0X, C000).	supportedOpcodes	decode, validate, getInstructionFormat
ProgramLoader	Reads a program file, validates lines (hex digits, blank lines, comments) and places the instructions in memory.	sourceFilePath, loadedInstructionCount	loadFromFile, parseLine, validateLine, loadIntoMemory
OutputDevice	Represents the screen; isolated from ordinary storage.	outputBuffer, lastOutput	display, clear, getOutput

2.3 Relationships

- **VoleMachine** composes CPU, Memory, RegisterFile, ProgramLoader and OutputDevice.
- **CPU** uses Memory (fetch and data access), RegisterFile (read/write) and InstructionDecoder (decode).
- **Memory** notifies OutputDevice when address FF is written. **ProgramLoader** loads the program into Memory.

2.4 Open questions I deliberately left for the AI review

I listed seven questions to be settled by comparing the AI designs: (1) should the CPU decode itself or use a decoder class; (2) should Memory call OutputDevice directly or through another abstraction; (3) composition or injected references; (4) separate loader or part of VoleMachine; (5) a dedicated Instruction type; (6) who owns validation and error handling; (7) are Register/MemoryCell classes justified?

2.5 PlantUML code

```
@startuml
skinparam classAttributeIconSize 0

class VoleMachine {
  - isInitialized: bool
  - cpu: CPU
  - memory: Memory
  - registerFile: RegisterFile
  - outputDevice: OutputDevice
  - programLoader: ProgramLoader
  + initialize(): void
  + reset(): void
  + loadProgram(filePath: string): bool
  + run(): void
  + step(): void
  + getMachineState(): string
}

class Memory {
  - cells: byte[256]
  - memorySize: int
  - outputAddress: byte
  + read(address: byte): byte
  + write(address: byte, value: byte): void
  + loadBytes(startAddress: byte, data): void
  + clear(): void
  + isValidAddress(address: int): bool
}

class RegisterFile {
  - registers: byte[16]
  - registerCount: int
  + read(registerIndex: int): byte
  + write(registerIndex: int, value: byte): void
  + clear(): void
  + isValidRegister(registerIndex: int): bool
}

class CPU {
  - programCounter: byte
  - instructionRegister: short
  - isHalted: bool
  + fetch(): short
  + decode(instruction: short)
  + execute(instruction: short): void
  + step(): void
  + run(): void
  + updateProgramCounter(): void
  + halt(): void
  + reset(): void
}

class InstructionDecoder {
  - supportedOpcodes
  + decode(instruction: short): Instruction
  + validate(instruction: short): bool
  + getInstructionFormat(opcode: byte): string
}

class ProgramLoader {
  - sourceFilePath: string
  - loadedInstructionCount: int
  + loadFromFile(filePath: string): bool
  + parseLine(line: string): Instruction
  + validateLine(line: string): bool
  + loadIntoMemory(instructions, startAddress): void
}

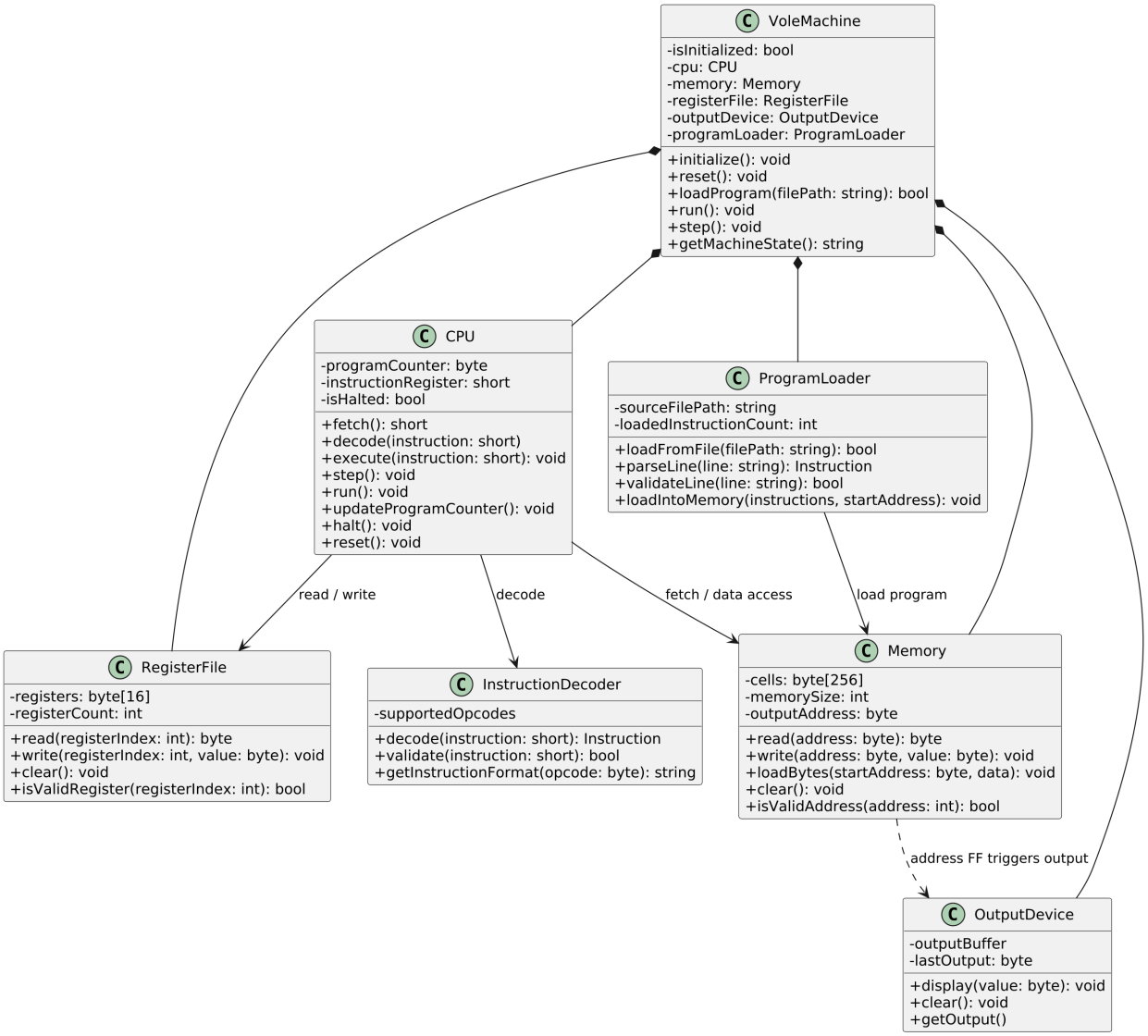
class OutputDevice {
  - outputBuffer
  - lastOutput: byte
  + display(value: byte): void
  + clear(): void
  + getOutput()
}

VoleMachine *-- CPU
VoleMachine *-- Memory
VoleMachine *-- RegisterFile
VoleMachine *-- ProgramLoader
VoleMachine *-- OutputDevice

CPU --> Memory : fetch / data access
CPU --> RegisterFile : read / write
CPU --> InstructionDecoder : decode
Memory ..> OutputDevice : address FF triggers output
ProgramLoader --> Memory : load program

@enduml
```

Figure 1: UML class diagram of my initial design



3. AI Designs

I gave each of three different AIs, **separately**, a description of the Vole machine, the required task and my initial design, then guided it in a chat to produce its best design and PlantUML. A follow-up prompt asked each AI to critically review its own first design and produce a revised one. The PlantUML and diagrams below are the **revised** designs saved in `ai-conversations/<ai>/<ai> - revised answer.md`. The full conversations and prompts are linked below and stored in the repository.

The strengths and weaknesses in this section were identified by comparing each design against the machine specification and against my initial design, with the help of the fourth AI (Section 4). I looked only at what each design actually contains.

3.1 Google Gemini

Chat: https://share.gemini.google/Cu4BA2LyB5Cs

Design. Machine (facade), CPU (which owns the RegisterFile), a stateless ALU, Memory with an `IMemoryListener` observer interface, DisplayDevice as a listener, Instruction + InstructionDecoder, Loader and a SimulatorException type.

Strong points

- Stateless **ALU** keeps arithmetic and bit operations (including float add and rotate) out of the CPU and easy to test.
- Explicit **halted** flag in the CPU gives the run loop a clean stop condition.
- Observer interface decouples Memory from a concrete display; exception types are named for errors.
- Correctly refuses per-cell and per-register classes.

Weak points

- **Instruction** has overlapping fields (r, s, t, x, y): x and y are the same nibbles as s and t, and some are always meaningless.
- The observer receives **every** memory write, so each listener must itself know that FF is special — the memory map leaks into the display.
- The diagram never connects DisplayDevice to Memory or Machine, so nothing registers the listener. Multiplicity "1..*" should be 0..*.
- Loader writes into Memory directly, so it can bypass the FF rule. CPU gets Memory through parameters while owning RegisterFile (inconsistent).
- Claims jumps must handle "alignment", which the specification does not say. Only the base exception is drawn, not the subclasses named in the text.

```
@startuml
skinparam classAttributeIconSize 0

interface IMemoryListener {
    + onByteWritten(address: uint8_t, value: uint8_t): void
}

class Machine {
    - running: bool
    + loadProgram(filepath: string): void
    + step(): void
    + run(): void
    + reset(): void
    + isHalted(): bool
}

class CPU {
    - pc: uint8_t
    - ir: uint16_t
    - halted: bool
    + fetch(memory: Memory): void
    + decode(): Instruction
    + execute(inst: Instruction, memory: Memory): void
    + step(memory: Memory): void
```

```
    + reset(): void
    + isHalted(): bool
}

class RegisterFile {
    - registers: std::array<uint8_t, 16>
    + read(index: uint8_t): uint8_t
    + write(index: uint8_t, value: uint8_t): void
    + reset(): void
}

class ALU {
    + addTwoComplement(a: uint8_t, b: uint8_t): uint8_t
    + addFloat(a: uint8_t, b: uint8_t): uint8_t
    + bitwiseOr(a: uint8_t, b: uint8_t): uint8_t
    + bitwiseAnd(a: uint8_t, b: uint8_t): uint8_t
    + bitwiseXor(a: uint8_t, b: uint8_t): uint8_t
    + rotateRight(val: uint8_t, n: uint8_t): uint8_t
}

class Memory {
    - storage: std::array<uint8_t, 256>
    - listeners: std::vector<IMemoryListener*>
    + readByte(address: uint8_t): uint8_t
    + writeByte(address: uint8_t, value: uint8_t): void
    + reset(): void
    + addListener(listener: IMemoryListener*): void
}

class Instruction {
    - opcode: uint8_t
    - r: uint8_t
    - s: uint8_t
    - t: uint8_t
    - x: uint8_t
    - y: uint8_t
    + getOpcode(): uint8_t
    + getR(): uint8_t
    + getS(): uint8_t
    + getT(): uint8_t
    + getX(): uint8_t
    + getY(): uint8_t
}

class InstructionDecoder {
    + decode(highByte: uint8_t, lowByte: uint8_t): Instruction
}

class Loader {
    + parseFile(filepath: string, memory: Memory): void
}

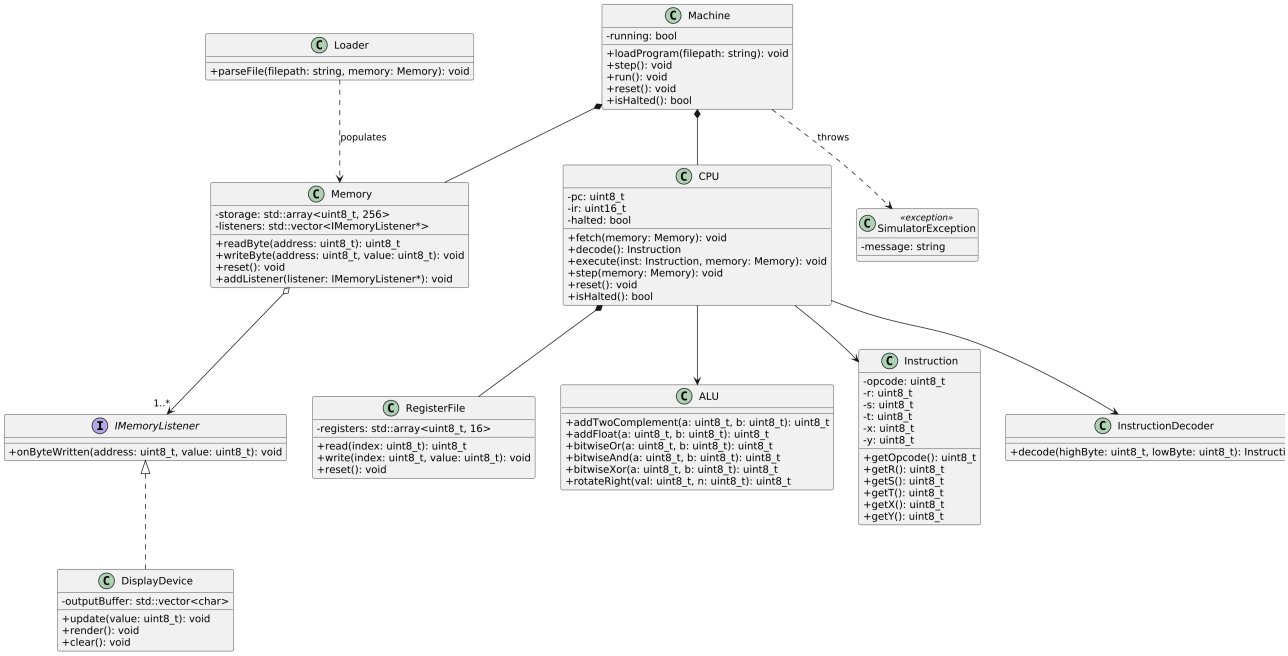
class DisplayDevice {
    - outputBuffer: std::vector<char>
    + update(value: uint8_t): void
    + render(): void
    + clear(): void
}

class SimulatorException <<exception>> {
    - message: string
}

Machine *-- CPU
Machine *-- Memory
CPU *-- RegisterFile
CPU --> ALU
CPU --> InstructionDecoder
CPU --> Instruction
Memory o--> "1..*" IMemoryListener
DisplayDevice ..up|> IMemoryListener
Loader ..> Memory : populates
Machine ..> SimulatorException : throws

@enduml
```

Figure 2: Google Gemini - revised design (rendered from its PlantUML)



3.2 OpenAI ChatGPT

Chat: https://chatgpt.com/share/6ade6ac8-aca0-83ea-8fd5-b08bdd2325d6

Design. Machine, CPU, Memory (holds the OutputDevice and intercepts FF), RegisterFile, InstructionDecoder, a small Instruction type with generic nibble fields `opcode, a, b, c`, ProgramLoader and OutputDevice. It explicitly argues against extra classes (Register, MemoryCell, ControlUnit, executor).

Strong points

- Removes the redundant **CPU to OutputDevice** dependency: all mapped output goes through Memory.
- Generic **a / b / c** nibble fields fit every instruction format without overlap.
- Three validation stages (file text, decoding, execution), byte-level fetch only, and **validate-the-whole-file-before-writing** loading.
- Explicit reset contracts, loader as a service (not a component), clear ownership with references.
- Spots the **FE/FF boundary problem** (an instruction at FE would need the screen cell FF) and warns about 8-bit PC+1 overflow.

Weak points

- All 12 opcodes are executed inside one big private CPU method, including bit-twiddling and the still-undefined float add.
- The loader still receives a Memory and writes into it; both Machine and CPU have `run()/step()` in the diagram although the text says only the CPU executes.
- No answer for how a fault reaches the caller; the read behaviour of address FF is left undecided.
- No read-only observers on Machine for testing and display.

```
@startuml
title Revised Vole Machine OOP Architecture

skinparam classAttributeIconSize 0
hide empty members

class Machine {
  - cpu : CPU
  - memory : Memory
  - registers : RegisterFile
  - output : OutputDevice
  + loadProgram(path : string, start : int) : void
  + run() : void
  + step() : void
  + resetCPU() : void
  + reset() : void
}

class CPU {
  - pc : uint8
  - ir : uint16
  - halted : bool
  - memory : Memory&
  - registers : RegisterFile&
  - decoder : InstructionDecoder
  + fetch() : void
  + step() : void
  + run() : void
  + reset() : void
  + isHalted() : bool
  + getPC() : uint8
  + getIR() : uint16
  - execute(i : Instruction) : void
}

class Memory {
  - cells : byte[256]
  - output : OutputDevice&
  + readByte(address : uint8) : byte
  + writeByte(address : uint8, value : byte) : void
  + clear() : void
  - validateAddress(address : int) : void
}
```

```
class RegisterFile {
  - registers : byte[16]
  + read(index : uint8) : byte
  + write(index : uint8, value : byte) : void
  + clear() : void
  - validateIndex(index : int) : void
}

class InstructionDecoder {
  + decode(word : uint16) : Instruction
  - validateFormat(opcode : uint8, a : uint8, b : uint8, c : uint8) : void
}

class Instruction {
  + opcode : uint8
  + a : uint8
  + b : uint8
  + c : uint8
}

class ProgramLoader {
  + parseFile(path : string) : vector<uint16>
  + parseLine(line : string) : optional<uint16>
  + loadInto(memory : Memory, path : string, start : int) : void
  - validateText(text : string) : void
}

class OutputDevice {
  - outputBuffer : string
  + write(value : byte) : void
  + display() : void
  + clear() : void
}

Machine *-- "1" CPU : owns
Machine *-- "1" Memory : owns
Machine *-- "1" RegisterFile : owns
Machine *-- "1" OutputDevice : owns

CPU --> "1" Memory : accesses
CPU --> "1" RegisterFile : reads/writes
CPU *-- "1" InstructionDecoder : uses

Memory --> "1" OutputDevice : mapped writes at FF

Machine ..> ProgramLoader : uses
ProgramLoader ..> Memory : loads validated bytes into
InstructionDecoder ..> Instruction : creates
CPU ..> Instruction : executes

note right of Machine
  Coordinates components.
  run() and step() delegate to CPU.
  resetCPU preserves the loaded program.
  reset clears the complete machine state.
end note

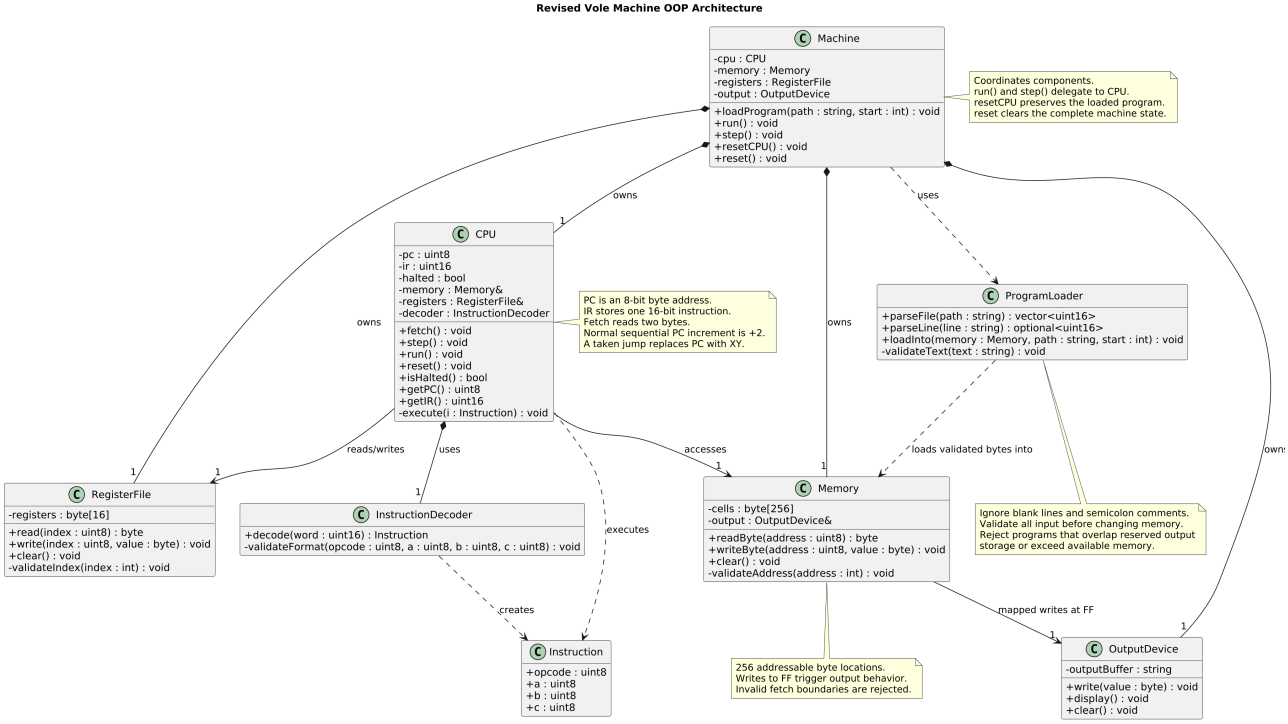
note right of CPU
  PC is an 8-bit byte address.
  IR stores one 16-bit instruction.
  Fetch reads two bytes.
  Normal sequential PC increment is +2.
  A taken jump replaces PC with XY.
end note

note bottom of Memory
  256 addressable byte locations.
  Writes to FF trigger output behavior.
  Invalid fetch boundaries are rejected.
end note

note bottom of ProgramLoader
  Ignore blank lines and semicolon comments.
  Validate all input before changing memory.
  Reject programs that overlap reserved output
  storage or exceed available memory.
end note

@enduml
```

Figure 3: OpenAI ChatGPT - revised design (rendered from its PlantUML)



3.3 DeepSeek

Chat: https://chat.deepseek.com/share/kqc985rblp5l3l6vh0

Design. VoleMachine facade, Memory, RegisterFile, CPU (owns a decoder and an `InstructionExecutor`), DecodedInstruction struct, ExecStatus/ExecResult, ProgramLoader with LoadResult/LoadError, MachineFault exception, and an OutputDevice interface with Console and MemoryBuffer implementations.

Strong points

- Best at stating the **unstated conventions**: PC counts bytes (+2), high byte first, FF read policy, rotate and reset order.
- Identifies the missing channel for halt / continue / fault between executor, CPU and machine.
- Line-numbered load errors; capacity check in the loader; honest trade-off note on folding the executor into the CPU.

Weak points

- **ExecStatus and ExecResult** both exist for one value and the diagram uses them inconsistently (CPU returns one, executor the other).
- The executor needs write access to the CPU (to change PC and halt), which weakens encapsulation; *halted* lives in VoleMachine, not CPU.
- "VoleMachine owns OutputDevice by value" cannot work when OutputDevice is an interface; two output implementations have no demonstrated need.
- The loader reports errors two ways (LoadResult and MachineFault). The DecodedInstruction fields s, t and xy overlap.
- States that 128 instructions fit in memory without noticing the FE/FF conflict with the screen.

```
@startuml
skinparam classAttributeIconSize 0

enum Opcode {
  Load
  LoadImmediate
  Store
  Move
  Add
  FloatAdd
  Or
  And
  Xor
  Rotate
  JumpEq
  Halt
  Invalid
}

enum ExecStatus {
  Continue
  Halt
}

struct DecodedInstruction {
  +Opcode op
  +uint8_t r
  +uint8_t s
  +uint8_t t
  +uint8_t xy
}

struct ExecResult {
  +ExecStatus status
}

struct LoadError {
  +int line
  +string message
}

struct LoadResult {
  +bool ok
  +vector<LoadError> errors
}

class MachineFault {
  +string message
```

```
}

class VoleMachine {
  -Memory memory
  -RegisterFile registers
  -CPU cpu
  -OutputDevice output
  -ProgramLoader loader
  -bool halted
  +loadProgram(path) : LoadResult
  +run()
  +step() : ExecStatus
  +reset()
  +getRegister(i) : uint8_t
  +getMemory(addr) : uint8_t
  +getPC() : uint8_t
  +getIR() : uint16_t
}

class Memory {
  -array<uint8_t,256> cells
  -OutputDevice* screen
  -uint8_t lastScreenByte
  +read(addr) : uint8_t
  +write(addr, byte)
  +reset()
}

class RegisterFile {
  -array<uint8_t,16> regs
  +read(index) : uint8_t
  +write(index, value)
  +reset()
}

class CPU {
  -uint8_t pc
  -uint16_t ir
  -Memory* memory
  -RegisterFile* registers
  -InstructionDecoder decoder
  -InstructionExecutor executor
  +step() : ExecStatus
  +fetch()
  +decode() : DecodedInstruction
  +execute(d) : ExecStatus
  +reset()
  +getPC() : uint8_t
  +getIR() : uint16_t
}

class InstructionDecoder {
  +decode(word) : DecodedInstruction
  -opcode(word) : uint8_t
  -nibble(word, n) : uint8_t
}

class InstructionExecutor {
  +execute(d, cpu, memory, registers) : ExecResult
  -opLoad(...)
  -opLoadImmediate(...)
  -opStore(...)
  -opMove(...)
  -opAdd(...)
  -opFloatAdd(...)
  -opOr(...)
  -opAnd(...)
  -opXor(...)
  -opRotate(...)
  -opJumpEq(...)
  -opHalt(...)
}

class ProgramLoader {
  +load(path, memory) : LoadResult
  -parseLine(line) : optional<uint16_t>
  -fitsInMemory(count) : bool
}

interface OutputDevice {
  +write(c : char)
  +clear()
  +getContents() : string
}

class ConsoleOutputDevice {
  +write(c : char)
  +clear()
  +getContents() : string
}

class MemoryBufferOutput {
  -string buffer
  +write(c : char)
```

```
  +clear()
  +getContents() : string
}

VoleMachine *-- Memory
VoleMachine *-- RegisterFile
VoleMachine *-- CPU
VoleMachine *-- OutputDevice
VoleMachine *-- ProgramLoader
VoleMachine ..> MachineFault : catches

CPU *-- InstructionDecoder
CPU *-- InstructionExecutor
CPU --> Memory
CPU --> RegisterFile
CPU ..> DecodedInstruction
CPU ..> ExecStatus

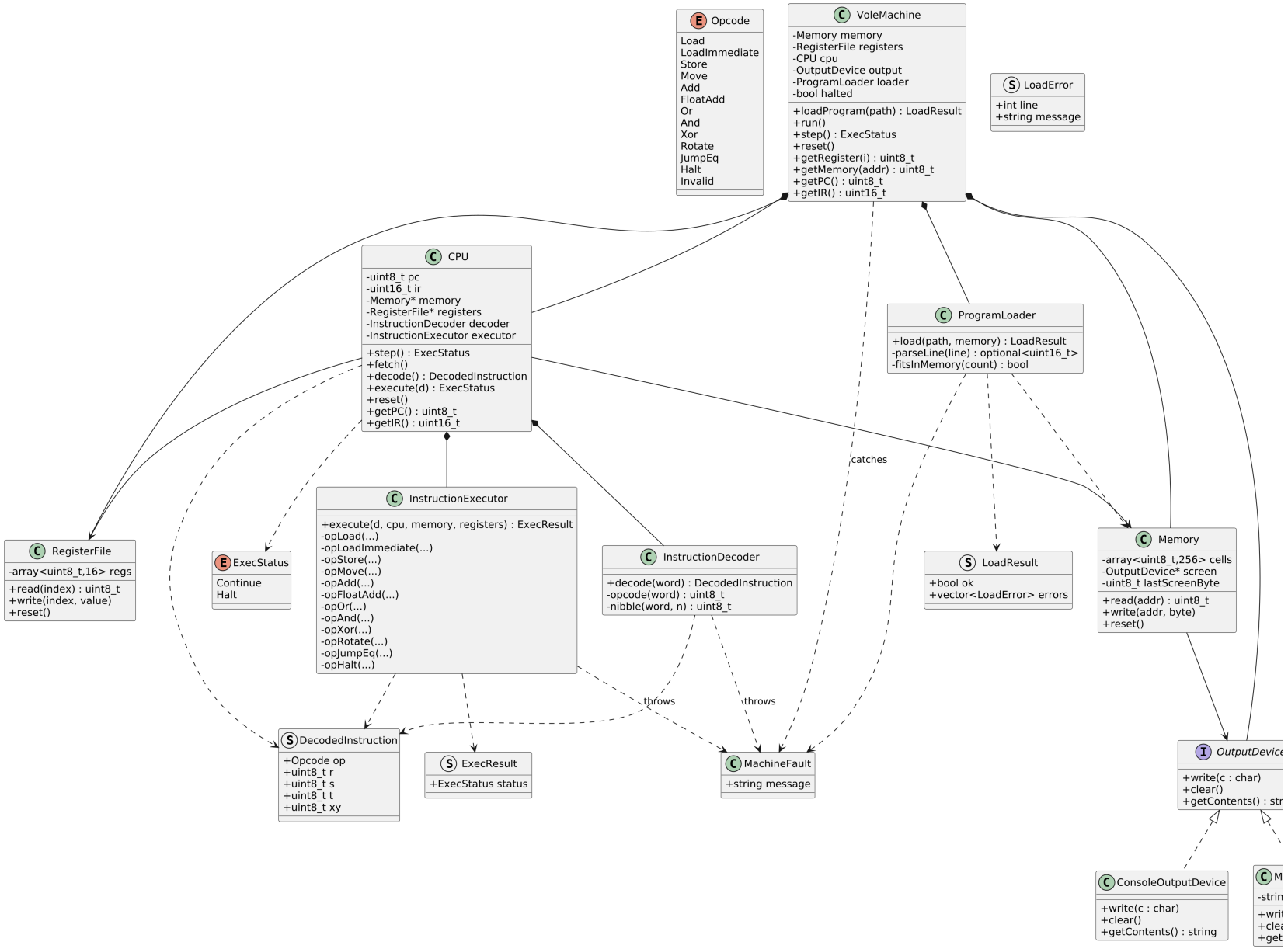
Memory --> OutputDevice
ProgramLoader ..> Memory
ProgramLoader ..> LoadResult
ProgramLoader ..> MachineFault

InstructionDecoder ..> DecodedInstruction
InstructionDecoder ..> MachineFault : throws
InstructionExecutor ..> DecodedInstruction
InstructionExecutor ..> ExecResult
InstructionExecutor ..> MachineFault : throws

OutputDevice <|.. ConsoleOutputDevice
OutputDevice <|.. MemoryBufferOutput

@enduml
```

Figure 4: DeepSeek - revised design (rendered from its PlantUML)



3.4 Comparison of the four designs

I judged designs by clarity of responsibility and correctness, not by number of classes. "Good", "Partial" and "Weak" are my assessments from the files.

Criterion	My initial	DeepSeek	Gemini	ChatGPT
Separation of responsibilities	Partial: decode/execute boundary unclear	Good, but executor entangled with CPU	Partial: odd Machine/CPU split	Good: clearest
Coupling	Partial: dangling output references	Partial: executor needs CPU&	Partial: listener knows FF	Good: lowest
CPU state, PC, IR, halt	Partial: no halt/error model	Partial: halt channel fixed, flag in wrong class	Good: halted in CPU	Good: halted, getPC/IR
Decoding and validation	Partial: overlapping methods	Good: decoder throws	Partial: overlapping fields	Good: three-stage model
Loading and errors	Partial	Good line errors; two channels	Partial: writes memory directly	Good: validate, then write
Output at FF	Partial: read/write undefined	Good: read policy decided	Partial: observer leaks FF	Partial: write via Memory; read undecided
Reset behaviour	Partial: overlapping	Good (reasoning weak)	Weak: minimal	Good: explicit contracts
Ownership and relationships	Partial	Partial: by-value interface	Partial: inconsistent	Good: consistent
Unneeded abstraction	Good: lean	Partial: 2 outputs, ExecResult	Partial: observer + ALU + exceptions	Good: leanest
Correctness against the machine	Partial	Mostly good	Partial: alignment claim	Mostly good

Which AI produced the best result?

ChatGPT's design was the strongest base: it is the leanest, has the lowest coupling, and was the most careful about not inventing behaviour. **DeepSeek** produced the best list of missing semantic details, and **Gemini** contributed the ALU idea and the exception types. The designs are similar in their core split (machine, CPU, memory, registers, decoder, loader, output) and in using arrays instead of per-cell classes. They differ mostly in how they deal with the screen at FF, how execution and halting are signalled, and how many extra layers they add. I therefore selected ChatGPT's design as the base for the final refinement.

4. Final Design

Chat: https://claude.ai/share/b2b07e58-df8e-465e-9e09-83bb4fb49743 (prompts and answers also saved in `ai-conversations/final revision/`)

4.1 How the final design was reached

I used a fourth AI, **Claude**, in two rounds.

- **Round 1 — comparative design review.** I gave Claude my initial design, the three AIs' revised designs and a five-phase prompt: understand the requirements, compare all designs against 13 criteria, resolve ten specific design decisions with alternatives and reasoning, produce the final architecture and PlantUML, and give report-ready evidence. I told it not to prefer a design just because it was elaborate and not to add unsupported features. Claude derived the final design from the comparison, using ChatGPT's structure as the base.
- **Round 2 — technical-correctness check.** I attached the official assignment specification and asked for one last review of instruction formats, PC/fetch behaviour, the FE/FF boundary, the justification of each class, relationship consistency and PlantUML validity, with every assumption labelled. This round found ten real problems (Section 4.7) and produced the corrected final diagram.

4.2 Final classes

Class	Responsibility	Main members
ConsoleUI	Menu loop, load/edit/run dialogs, shows memory matrix, IR, PC, registers and screen; "Enter for next step".	start, showMenu, handleLoad, handleEdit, handleRun, displayState; owns a VoleMachine
VoleMachine	Facade that owns the hardware; loads programs; steps/runs; catches runtime faults.	loadProgram, setRegister, setMemory, step, run(maxSteps), reset, read-only getters
CPU	The only class that performs fetch-decode-execute; owns PC, IR and status (RUNNING / HALTED / FAULTED).	step, startAt, reset, markFaulted, getPC, getIR, getStatus; private fetch, execute
InstructionDecoder	Stateless: turns a 16-bit word into a validated Instruction; throws on invalid ones.	static decode, checkFormat
Instruction	Small value type: opcode (enum) and three nibbles a, b, c.	opcode, a, b, c, xy()
ALU	Stateless 8-bit arithmetic/logic: two's-complement add, float add, OR, AND, XOR, rotate right.	static add / or / and / xor / rotateRight
Memory	256 bytes; range checks; the only class that knows FF is the screen.	read, write, canLoad, loadBytes, reset; constants SIZE, OUTPUT_ADDRESS
RegisterFile	16 byte registers with index checks.	read, write, reset; constant COUNT
ProgramLoader	Stateless: reads the file, parses lines, validates every word, returns the words.	static loadFile, parseLine
OutputDevice	The screen: records every byte written; display format is left to the UI.	write, clear, getHistory
SimulatorError	One exception type carrying a message (invalid instruction, bad address, bad file).	what

Plus two enums: **Opcode** (LOAD_MEM … HALT) and **CpuStatus**.

4.3 Instruction field mapping (checked against the specification examples)

Format	Meaning	a	b	c	Decoder rule
`1RXY 2RXY 3RXY`	load / load-immediate / store	R	X	Y	XY = (b,c)
`40RS`	move R to S	0	R (source)	S (dest.)	a must be 0
`5RST ... 9RST`	add, float add, OR, AND, XOR	R (result)	S	T	none
`AR0X`	rotate R right X times	R	0	X	b must be 0
`BRXY`	jump if R == R0	R	X	Y	none
`C000`	halt	0	0	0	all must be 0

Opcodes 0 and D-F are not in the instruction table and are therefore invalid. Rejecting non-zero "reserved" digits is my assumption (4.8). Note that `B0XY` compares R0 with itself, so it always jumps.

4.4 Execution workflow

1. **Load.** ConsoleUI asks for a file and an even start address. `VoleMachine::loadProgram` calls `ProgramLoader::loadFile`, which skips blank lines and lines starting with ";", reads the first four hex digits of each remaining line (the rest is ignored) and runs every word through `InstructionDecoder`. Any bad line or invalid instruction rejects the whole program before anything is changed.
2. The machine checks that the start is even and below FF and that the program fits (`Memory::canLoad`), writes the bytes (high byte first, no screen output) and calls `cpu.startAt(start)`, which sets PC, clears IR and sets RUNNING.
3. **Run or step.** `run()` repeats `step()` while the CPU is RUNNING; the UI shows the state after the run, or after each step in step mode.
4. **One step.** fetch two bytes into IR and add 2 to PC; decode and validate; execute. A taken `BRXY` overwrites PC with XY; `C000` sets HALTED; a store to FF is forwarded by Memory to the OutputDevice.
5. **Faults.** Any `SimulatorError` during a step is caught in VoleMachine, which marks the CPU FAULTED and keeps the message; the state stays inspectable. The UI then offers to exit or load another program.

4.5 Error handling

Problem	Detected by	Result
Unreadable file; a line that does not start with four hex digits	ProgramLoader	SimulatorError (with line number); nothing loaded
Invalid opcode or bad reserved digit in a program file	InstructionDecoder (called by the loader)	Program rejected, nothing loaded
Odd or too-large start address; program does not fit; would touch FF	VoleMachine / Memory::canLoad	SimulatorError; nothing loaded
Invalid instruction met at run time (e.g. after the user edits memory)	InstructionDecoder	CPU becomes FAULTED
Address outside 0-255, register index above 15, fetch at PC beyond FD	Memory / RegisterFile / CPU	CPU becomes FAULTED

4.6 Key design decisions

Decision	Alternatives considered	Choice and reason
Separate decoding from execution?	CPU does both; separate executor class (DeepSeek)	Decoder separate (pure and testable); execution stays in the CPU because it must change PC and halt state. An executor would need write access to the CPU.
Instruction data structure	Raw 16-bit word; class per opcode; named fields per format	One small value type {opcode, a, b, c}: fits every format with no overlapping fields; a hierarchy is not justified for 12 opcodes.
Memory and registers	One object per cell/register	Arrays wrapped in Memory and RegisterFile: cells have identical behaviour and no per-cell data.
Writing to FF	CPU checks FF; observer on Memory (Gemini); Memory forwards to OutputDevice	Memory forwards: it is the only class that knows the mapping. An observer makes every listener re-check the address.
Loader independence	Part of VoleMachine; loader writes memory	Stateless service that only returns validated words; it cannot bypass the FF rule and is easy to test.
Ownership	Everything owned by CPU; shared pointers	VoleMachine owns OutputDevice, Memory, RegisterFile and CPU; CPU and Memory keep non-owning references.
Errors	Result structs / codes; exception hierarchy	One exception type: no caller needs to tell error kinds apart. VoleMachine catches runtime faults.
PC, jumps, halt	PC counts instructions; halt as a special value	PC is a byte address, +2 at fetch; a jump overwrites it; halt is a CPU status.
Reset	Reset everything on every load	Loading must not wipe memory and registers (the spec allows several programs and manual edits). reset() clears everything on request. Memory::reset never prints to the screen.
Extra classes	Register, MemoryCell, ControlUnit, executor, observer interface	Rejected. Added only ConsoleUI (the spec requires menu and display), Instruction, and ALU (justified in 4.8).

4.7 Corrections made after checking the official specification

#	Problem found in the first final design	Fix
1	No class for the menu and display required by the spec.	Added ConsoleUI.
2	The loader checked syntax only; the spec says programs with invalid instructions must be rejected.	The loader validates every word with the decoder before anything is stored. The run-time check stays because memory can be edited.
3	loadProgram wiped memory and registers.	Loading keeps them; it writes the bytes and calls cpu.startAt(start).
4	Even start address (below FF) not enforced.	loadProgram rejects odd or too-large starts.
5	No way to overwrite a register or memory cell (required).	Added setRegister and setMemory.
6	OutputDevice echoed to a stream while running; the spec shows the screen after the run or step.	OutputDevice only records bytes; ConsoleUI displays them.
7	Four exception classes, none needed.	One SimulatorError.
8	Loader required nothing after four digits except comments; the spec says the rest of the line is ignored.	Only the first four hex digits matter; blank and ";" lines are skipped.
9	Both CPU and loader need the decoder.	Decoder made static and stateless; both depend on it.
10	restart() and setPC() were unnecessary or hid a side effect.	Removed restart; replaced setPC with startAt.

4.8 Specified versus assumed

The assignment text is the authority. Where it is silent I labelled the behaviour an **assumption** rather than a requirement.

Behaviour	Status
16 8-bit registers; 256 cells; two-byte instructions; PC +2; jump copies XY into PC; BRXY compares R with R0	Specified
Program loaded at an even address below FF; blank and ";" lines ignored; rest of line after four hex digits ignored; invalid programs rejected	Specified
Writing to FF writes to the screen	Specified
A store to FF also keeps the byte in cell FF; reading FF returns it	Assumption
No program byte may be placed at FF; a fetch needs PC <= FD (so FE is a legal start address by the text, but cannot hold a whole instruction)	Assumption (one constant / one check)
Integer add wraps modulo 256; PC after a halt is the address after C000	Assumption
Float add uses Brookshear’s 8-bit float format (sign, 3-bit excess-4 exponent, 4-bit mantissa)	Assumption (format not given in the assignment)
Non-zero reserved digits in 40RS, AR0X, C000 are invalid; odd jump targets are allowed; loading clears the screen history	Assumptions
ALU class, Instruction type, ConsoleUI and the maxSteps guard on run()	Design choices (not required by the spec)

The ALU is the most debatable class. It is not required; it is kept because float add is non-trivial and benefits from isolated testing, and it keeps bit-twiddling out of the CPU’s execute method. It can be folded into the CPU with no other change.

4.9 Why this is the best design

- **Visibly derived from the evidence.** It starts from ChatGPT's lean structure, adds DeepSeek's explicit conventions and fault handling, and Gemini's ALU, and rejects the extra layers (executor, observer, result structs, two output devices) that solved no demonstrated problem.
- **Every responsibility has one owner.** Only Memory knows about FF; only the CPU executes; only the decoder interprets words; the loader never touches memory; the UI is the only class that prints.
- **Matches the specification.** It covers the menu, even-address loading, editing of registers and memory, step/run modes, display of memory/IR/PC/registers/screen, and rejection of invalid programs.
- **Consistent ownership and relationships.** Composition for what VoleMachine owns; non-owning references elsewhere; stateless helpers (decoder, ALU, loader) used with dependencies only.
- **Small and testable.** Ten classes and two enums; decoder, ALU and loader can be tested without a machine; memory and CPU can be tested with a recording OutputDevice.

How it improves on each earlier design

- **My initial design:** adds a decoded Instruction, explicit halt/fault states, a UI class, stateless loader, constants as constants (not attributes), removes initialize/isInitialized and dangling output references.
- **DeepSeek:** keeps its conventions but drops the executor and the duplicate ExecResult/ExecStatus; one error channel; no by-value interface.
- **Gemini:** keeps the ALU, replaces the observer with a direct Memory to OutputDevice link, and uses non-overlapping instruction fields; the loader no longer writes memory.
- **ChatGPT:** keeps its structure and rules but decides the FF policy, adds the ALU and a fault path to the caller, and removes the loader's memory access.

4.10 Points to confirm with the instructor

- The sample program in the assignment ends with `3200` and the comment "Store register 2 into memory[00] (screen output)", but the instruction table says the screen is at **FF**. Should the sample read `32FF`? (My own programs use FF.)
- How should screen bytes be displayed: decimal, hex or ASCII? (The sample says the result is "12", which is stored as 0C.)
- May a program occupy FF, and is a start address of FE legal?
- What is the exact 8-bit float format for 6RST?
- Should reserved digits in 40RS, AR0X and C000 be strictly validated; are odd jump targets allowed; should loading clear the screen, memory or registers?

4.11 Final PlantUML code

```
@startuml
title Final Vole Machine Architecture (spec-checked)
skinparam classAttributeIconSize 0
hide empty members

enum Opcode {
  LOAD_MEM = 1
  LOAD_IMM = 2
  STORE = 3
  MOVE = 4
  ADD_INT = 5
  ADD_FLOAT = 6
  OR = 7
  AND = 8
  XOR = 9
  ROTATE = 10
  JUMP_EQ = 11
  HALT = 12
}

enum CpuStatus {
  RUNNING
  HALTED
  FAULTED
}

class ConsoleUI {
  - machine : VoleMachine
  + start() : void
  - showMenu() : void
  - handleLoad() : void
  - handleEdit() : void
  - handleRun() : void
  - displayState() : void
}

class VoleMachine {
  - output : OutputDevice
  - memory : Memory
  - registers : RegisterFile
  - cpu : CPU
  - lastError : string
  + VoleMachine()
  + loadProgram(path : string, start : uint8) : void
  + setRegister(index : int, value : uint8) : void
  + setMemory(address : int, value : uint8) : void
  + step() : CpuStatus
  + run(maxSteps : int = 0) : CpuStatus
  + reset() : void
  + getLastError() : string
  + getCpu() : const CPU&
  + getMemory() : const Memory&
  + getRegisters() : const RegisterFile&
  + getOutput() : const OutputDevice&
}

class CPU {
  - pc : uint8
  - ir : uint16
  - status : CpuStatus
  - memory : Memory&
  - registers : RegisterFile&
  + CPU(m : Memory&, r : RegisterFile&)
  + step() : void
  + startAt(address : uint8) : void
  + reset() : void
  + markFaulted() : void
  + getPC() : uint8
  + getIR() : uint16
  + getStatus() : CpuStatus
  - fetch() : void
  - execute(i : Instruction) : void
}

class InstructionDecoder <<stateless>> {
  + {static} decode(word : uint16) : Instruction
  - {static} checkFormat(i : Instruction) : void
}

class Instruction <<value>> {
  + opcode : Opcode
  + a : uint8
  + b : uint8
  + c : uint8
  + xy() : uint8
}

class ALU <<stateless>> {
  + {static} addTwosComplement(x : uint8, y : uint8) : uint8
  + {static} addFloat(x : uint8, y : uint8) : uint8
  + {static} bitOr(x : uint8, y : uint8) : uint8
```

```
  + {static} bitAnd(x : uint8, y : uint8) : uint8
  + {static} bitXor(x : uint8, y : uint8) : uint8
  + {static} rotateRight(v : uint8, count : uint8) : uint8
}

class Memory {
  + {static} SIZE : int = 256
  + {static} OUTPUT_ADDRESS : int = 0xFF
  - cells : array<uint8, 256>
  - output : OutputDevice&
  + Memory(out : OutputDevice&)
  + read(address : int) : uint8
  + write(address : int, value : uint8) : void
  + canLoad(start : int, byteCount : int) : bool
  + loadBytes(start : int, data : vector<uint8>) : void
  + reset() : void
}

class RegisterFile {
  + {static} COUNT : int = 16
  - regs : array<uint8, 16>
  + read(index : int) : uint8
  + write(index : int, value : uint8) : void
  + reset() : void
}

class ProgramLoader <<stateless>> {
  + {static} loadFile(path : string) : vector<uint16>
  - {static} parseLine(line : string, lineNo : int) : optional<uint16>
}

class OutputDevice {
  - history : vector<uint8>
  + write(value : uint8) : void
  + clear() : void
  + getHistory() : vector<uint8>
}

class SimulatorError <<exception>> {
  + what() : string
}

ConsoleUI *-- "1" VoleMachine

VoleMachine *-- "1" OutputDevice
VoleMachine *-- "1" Memory
VoleMachine *-- "1" RegisterFile
VoleMachine *-- "1" CPU
VoleMachine ..> ProgramLoader : uses
VoleMachine ..> SimulatorError : throws / catches

CPU --> "1" Memory : reads/writes bytes
CPU --> "1" RegisterFile : reads/writes
CPU ..> InstructionDecoder : uses
CPU ..> ALU : uses
CPU ..> Instruction : executes
CPU ..> CpuStatus

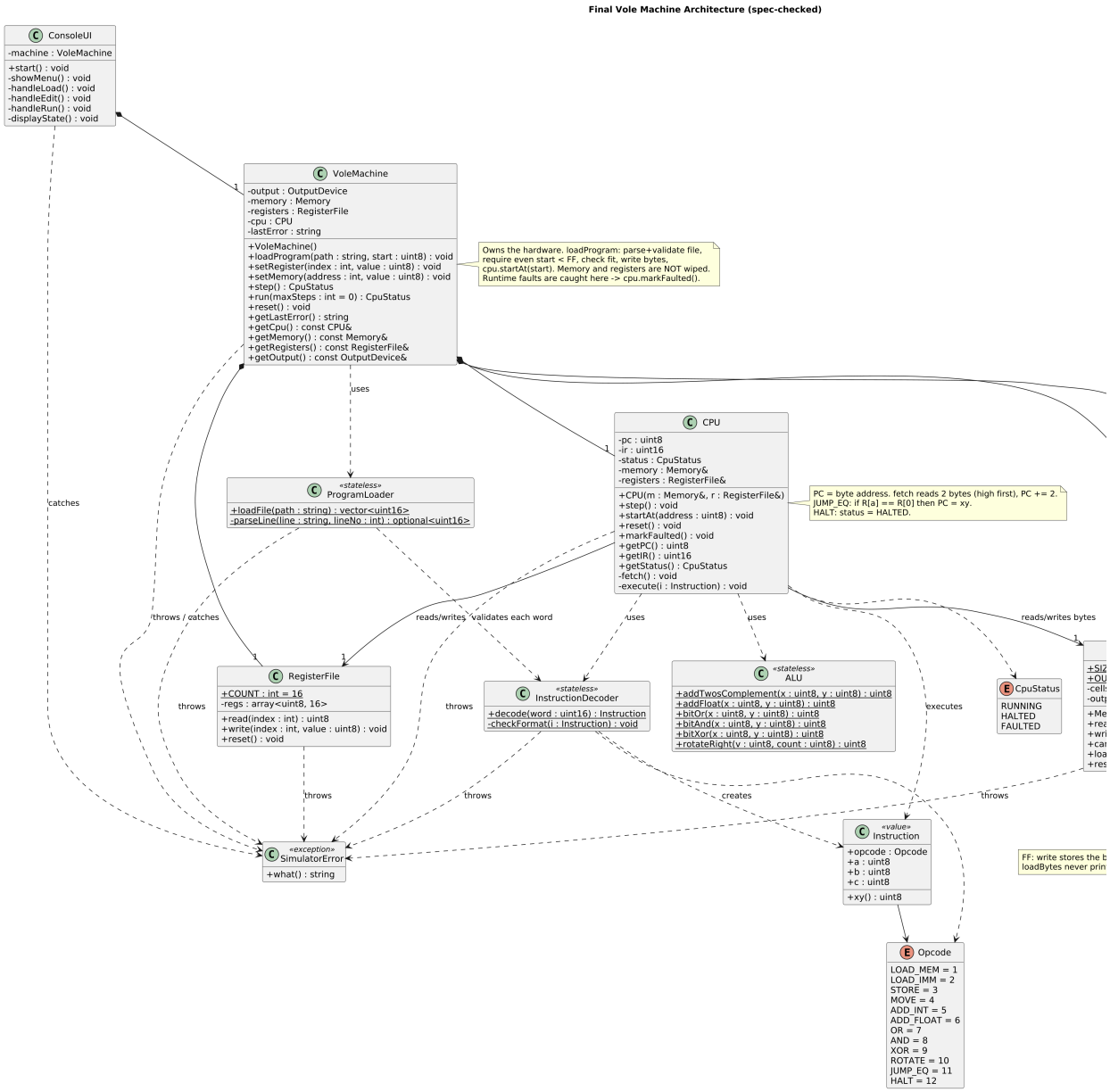
ProgramLoader ..> InstructionDecoder : validates each word
InstructionDecoder ..> Instruction : creates
InstructionDecoder ..> Opcode
Instruction --> Opcode

Memory --> "1" OutputDevice : write at FF

ConsoleUI ..> SimulatorError : catches
CPU ..> SimulatorError : throws
Memory ..> SimulatorError : throws
RegisterFile ..> SimulatorError : throws
ProgramLoader ..> SimulatorError : throws
InstructionDecoder ..> SimulatorError : throws

note right of VoleMachine
  Owns the hardware. loadProgram: parse+validate file,
  require even start < FF, check fit, write bytes,
  cpu.startAt(start). Memory and registers are NOT wiped.
  Runtime faults are caught here -> cpu.markFaulted().
end note
note right of CPU
  PC = byte address. fetch reads 2 bytes (high first), PC += 2.
  JUMP_EQ: if R[a] == R[0] then PC = xy.
  HALT: status = HALTED.
end note
note bottom of Memory
  FF: write stores the byte AND appends it to the screen.
  loadBytes never prints and never touches FF.
end note
@enduml
```

Figure 5: Final UML class diagram of the Vole machine simulator



4.12 What I learned

- Different AIs make different trade-offs from the same prompt. Comparing them made me justify each class instead of accepting the first answer.
- More classes is not a better design. The best candidate was the leanest, and several elaborate ideas (executor, observer, result structs) added coupling without solving a real problem.
- Reading the official specification last would have been a mistake: checking against it found ten real problems, including a missing UI class and loading rules I had wrong.
- Unstated conventions (PC units, byte order, what reading FF returns, FE/FF boundary) matter as much as the classes; it is better to label an assumption than to invent a requirement.
- Even the AI’s own claims need checking: I verified both of my sample programs against the instruction set before using them as test cases, and the tooling checks (PlantUML) caught syntax mistakes the text did not.

5. Examples

Two sample Vole programs written for evaluating the simulator. Both print to the screen by storing to **FF**, use a loop controlled by `BRXY`, and rely on `B0XY` (R0 compared with itself) as an unconditional jump. I verified each one by hand-simulating it against the instruction table, and both behave as described.

5.1 Program 1 — counter from 0 to 10

Address	Hex	Meaning
00	`200A`	R0 = 0A (target, 10)
02	`2100`	R1 = 00 (current count)
04	`2201`	R2 = 01 (addend)
06	`31FF`	store R1 to FF: show the current count on the screen (loop start)
08	`B110`	if R1 == R0 (count reached 10) jump to 10 (halt)
0A	`5312`	R3 = R1 + R2
0C	`4031`	move R3 to R1 (update the count)
0E	`B006`	R0 == R0 always, so jump back to 06
10	`C000`	halt

The three variables (target, count, addend) are initialised once before the loop. Each pass shows the count, tests it against the target, adds one and loops. **Expected screen output:** `00 01 02 03 04 05 06 07 08 09 0A` (11 values). It halts after 56 instruction steps with PC = 12.

```
200A
2100
2201
31FF
B110
5312
4031
B006
C000
```

5.2 Program 2 — reverse counter from 10 to 0

Address	Hex	Meaning
00	`2000`	R0 = 00 (target, 0)
02	`210A`	R1 = 0A (current count, 10)
04	`22FF`	R2 = FF (-1 in two’s complement)
06	`31FF`	store R1 to FF (screen)
08	`B110`	if R1 == R0 (reached 0) jump to 10 (halt)
0A	`5312`	R3 = R1 + R2, i.e. R1 - 1 (8-bit two’s-complement add)
0C	`4031`	move R3 to R1
0E	`B006`	unconditional jump back to 06
10	`C000`	halt

The machine has no subtract instruction, but adding FF (which represents -1 in 8-bit two’s complement) subtracts one. **Expected screen output:** `0A 09 08 07 06 05 04 03 02 01 00` (11 values). It halts after 56 steps with PC = 12.

```
2000
210A
22FF
31FF
B110
5312
```

```
4031
B006
C000
```

What they exercise: load-immediate, store to the memory-mapped screen, register move, two’s-complement addition, conditional and unconditional jumps, halt, the program loader format, and an exit through a conditional jump to the halt instruction.

6. Video Link

Presentation video (about 10 minutes) discussing the three AI designs, which AI produced the best results, and how similar or different they are:

https://drive.google.com/drive/folders/1kHbSSZJQRunnAbgXUU8N98l1X9Rz36o9?usp=drive_link

Appendix A: Repository structure

Repository: https://github.com/Mr-Youssef-Sherif/vole-machine

```
Vole machine/
  CS213-2026-2027-Assignment2-V4.0.pdf        (assignment specification)
  initial design/    Initial_Design.md, UML.md, UML.png
  ai-conversations/
    Initial Prompt.md, Follow-up Prompt.md, AI Chat References.md
    openai-chatgpt-conversation/   initial answer, revised answer, UML-design.md/.png
    google-gemini/                 initial answer, revised answer, UML-design.md/.png
    deepseek/                      initial answer, revised answer, UML-design.md/.png
    final revision/                Claude prompt, Claude initial response,
                                   Claude revision prompt, Claude revised response (FINAL),
                                   UML.md, UML.png
  programs/          program-1.txt, program-2.txt, Programs Describtion.md
  report/            this report
```

Appendix B: References

- J. Glenn Brookshear, *Computer Science: An Overview*, 11th edition, pp. 73-93 (the Vole machine).
- CS213 Programming II, Assignment 2, Version 4.0, Cairo University, Faculty of Computers and Artificial Intelligence, 2026-2027.
- PlantUML, `https://www.plantuml.com`, used to render every diagram in this report.